TOWSON UNIVERSITY
COLLEGE OF GRADUATE STUDIES

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE EDUCATION

by

Luis Dale Gascon

A thesis
Presented to the faculty
of Towson University
in partial fulfillment
of the requirements for the degree
Master of Science
Department of Computer and Information Sciences

Towson University,
Towson, Maryland
21252

December 2026

Docusign Envelope ID: 24F1B278-468C-4F3E-91E7-E942561CC002

TOWSON UNIVERSITY | Office of Graduate Studies

Thesis Committee Approval Form

Student's Name

Luis Dale Gascon

Chairperson, Thesis Committee DocuSigned by: Sajidul Islam A1834C6028884BD... (Signature) Md Sajidul Islam Sajid (Typed name)

Note: Please attach a description of the affiliation and credentials of any non- Towson University members of the Committee, and the members' *curriculum vitae.*

Member Signed by: Sidd Kaza 1C79FD345A714C7... (Signature) Sidd Kaza (Typed name)

Member DocuSigned by: Blair Taylor F736E4AFF5A443D... (Signature) Blair Taylor (Typed name)

Approved by

Graduate Program Director

Signed by: Eiying Tang C6D51D455F89448... (Signature) 02/02/26 (Date)

Department Chairperson

DocuSigned by: Michael McGuire 4FA74552EDAD4F1... (Signature) 02/02/26 (Date)

☒ Graduate Faculty Status verified for all committee members

Dean of Graduate Studies

Signed by: David R Ownby 0109A385FE4043B... (Signature) 02/03/26 (Date)

Acknowledgments

I would like to express my sincere gratitude to Dr. Taylor and Dr. Kaza for their continuous support throughout my graduate studies. I will always be thankful for the opportunities that they have provided.

Special thanks go to my thesis advisor, Dr. Sajid, for generously agreeing to advise this work on short notice and for dedicating his time to reviewing my drafts under a tight deadline. His insightful suggestions greatly strengthened this thesis, and I deeply appreciate his willingness to invest the time and effort to engage with such an esoteric subject matter.

I would also like to thank the student volunteers, whom I endearingly referred to as "my lab rats" for participating in the onboarding experiments. Their time, patience, and candid feedback were invaluable to evaluating the practical ergonomics of this work.

I'd also like to thank the Outdoor Adventure Club climbing walls for the friendships I've made there that kept me sane through my graduate program.

Finally, I am deeply grateful to my parents for their unconditional belief in my potential and their sacrifices on my behalf. I wouldn't be where I am today without their steadfast love and encouragement.

Abstract

AI-ASSISTED ADOPTION OF NIX FLAKES FOR REPRODUCIBLE DEVELOPMENT ENVIRONMENTS IN COMPUTER SCIENCE EDUCATION

Luis Dale Gascon

Inconsistent development environments pose substantial friction for software engineers and development teams, frequently consuming days of onboarding time and shifting engineering focus from feature delivery to environment debugging. In computer science education, this challenge is acutely amplified. When students must imperatively configure their personal machines for diverse coursework, valuable lecture and lab hours are often sacrificed to troubleshooting dependency conflicts. The myriad combination of student hardware, operating systems, and local configuration states turns instructors and teaching assistants into ad-hoc technical support providers, creating severe barriers to student engagement and learning.

This thesis proposes an automated workflow that leverages agentic large language models (LLMs) to synthesize reproducible development environments using Nix Flakes from instructors' natural language requirements or to infer requirements from a project's source code. Because environment synthesis is a recurrent task across semesters, we designed and implemented a specialized custom agent skill that provides an agent with domain-specific flake templates, structured metadata, and automated validation heuristics to repeatedly perform the task with the proper context. To prevent package hallucination within the massive Nixpkgs ecosystem, the

agent reaches out to a Model Context Protocol (MCP) server to retrieve real-time package metadata and configuration schemas directly from NixOS search indexes. Synthesized flakes are verified through a deterministic evaluation pipeline using native Nix tooling, enabling the agent to autonomously interpret evaluation error traces to self-correct invalid specifications. Furthermore, this study evaluates student onboarding ergonomics by comparing declarative environment activation via Nix Flakes against traditional imperative installation, measuring the time required to get students up and running with a functional development environment alongside subjective cognitive effort captured from student feedback to determine whether introducing Nix into a course is worthwhile in practice. Finally, this thesis examines the security implications of using nix, analyzing how binary caches can serve as a potential attack vector and evaluating the security measures in place both to secure binary caches and to mitigate supply-chain attacks.

Table of Contents

List of Tables ix

List of Figures x

List of Listings xi

Chapter I. Introduction 1

A. The Reproducibility Issue in Computer Science Education 1

B. Virtualization versus Declarative Configuration: The Architectural Dilemma 2

C. AI-Assisted Synthesis: Overcoming the Usability Barrier 4

D. Research Objectives and Contributions 6

E. Organization of the Thesis 8

Chapter II. Background and Literature Review 9

A. Related Work 9

1) Standardizing Educational Environments via Virtual Machines and Containers 9

2) Learning Opportunity Provided from Managing Systems 11

3) Declarative and Functional Package Management in Education 11

4) Synthesis of Infrastructure as Code Using Large Language Models 13

5) Agentic Skills and Tool-Augmented Assistance 14

B. Nix Foundations 16

1) Declarative Packages and Nixpkgs 16
2) The Nix Store and Dependency Closures 17
3) Flakes, Locked Inputs, and Development Shells 18
4) Realization, Binary Caches, and Trust 21
C. Summary of Research Gaps 22
Chapter III. Research Questions and Proposed Architecture 25
A. Research Questions 25
B. Proposed Solution: An agent skill for AI-Assisted Nix Flake Synthesis 26
Chapter IV. Experimental Methodology and Test Setup 29
A. Agentic Flake Synthesis Evaluation Protocol 29
1) Benchmark Dataset Curation and Curriculum Tracks 30
2) Agent Harness, Model Configurations, and Tooling Conditions 31
3) Three-Stage Validation Pipeline and Pass@K Formulation 34
B. Clean-Environment Reproducibility Protocol 35
1) Testing on 35
2) Guest Replay Pipeline and Failure Taxonomy 36
C. Student Onboarding Ergonomics Protocol 37
D. Experimental Testbed and System Infrastructure 38
Chapter V. Empirical Results and Analysis 40
A. Flake Synthesis Accuracy and Model Comparison 40
B. Tool Invocation and Telemetry Analysis 41
1) Native Nix CLI Discovery versus MCP Retrieval 43

C. Base OS Reproducibility Results 46
D. Student Onboarding Ergonomics Results 47
1) Installation of the Rust Compiler 47
a) Student A 47
b) Student B 49
2) Jupyter Notebook 49
3) Full Stack Environment 49
Chapter VI. Conclusion 51
A. Summary of Contributions 51
B. Synthesis of Research Questions 52
C. Discussion: Operational Constraints and Deployment Limitations ... 53
1) Adopting Nix 53
2) Operational Availability and Maintenance Conditions 54
3) Security Considerations and Binary Cache Trust Boundaries .. 55
D. Limitations and Future Work 56
Appendix A Appendix Title Here 58
A. Disclosure of LLM use 58
1) AI-Generated Artifacts 58
2) AI-Generated Input 62
Appendix B Appendix Title Here 64
References 65

List of Tables

Table I Comparison of Development Environment Management Approaches 24

Table II Host Machine Hardware and System Environment 38

Table III Flake Synthesis Accuracy Across Models and MCP Conditions 42

Table IV Mean Trial Duration and Token Consumption Across Models and MCP Conditions 42

Table V Package Precision and Over-Provisioning Ratio Across Passing Trials 42

Table VI MCP Tool Invocation Comparison in MCP-Enabled Evaluations 46

Table VII Onboarding Results for Rust Toolchain Setup 48

Table VIII Onboarding Results for Full Stack Environment Setup 50

List of Figures

Fig. 1 End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline 28

List of Listings

Listing 1 Example of a flake file 18

Listing 2 `flake.lock` `nixpkgs` Attributes 20

Listing 3 Input Data Format 30

Listing 4 Custom Skill Filetree Structure 31

Listing 5 assets/flake-devshell.template.md template 32

Listing 6 Natural Language Instruction Benchmark Inputs 62

Chapter I.

Introduction

A. The Reproducibility Issue in Computer Science Education

Software Configuration Management continues to be a critical component of modern software engineering, ensuring consistency and deterministic reproducibility across development, testing, and deployment cycles [1–3]. In commercial software engineering, misconfigured developer machines frequently trigger the classic "works on my machine" syndrome, where subtle variances in operating system libraries, compiler revisions, or environment variables introduce non-deterministic bugs. Enterprise organizations systematically mitigate this friction through dedicated platform teams, standardized developer virtual machines, and automated continuous integration pipelines. In contrast, academic computer science programs either expect students to install and configure the required toolchains on their own, or provide pre-configured centralized servers, virtual machines, or containers that each introduce distinct operational and resource trade-offs [4–6].

In modern computer science courses, students arrive in classrooms with a highly heterogeneous array of personal devices spanning macOS, disparate Linux distributions, and Microsoft Windows [5, 6]. Introductory and advanced courses alike demand specialized toolchains, language runtimes, software development kits (SDKs), and external libraries. Historically, instructors have relied on imperative, manual installation guides distributed during the opening weeks of the semester [5, 7].

This approach invariably consumes substantial lecture and laboratory hours as teaching assistants and faculty diagnose idiosyncratic system failures, operating as impromptu technical support staff rather than focusing on foundational computer science concepts [5, 6].

Holland et al. [8] characterize this manual tinkering with a system's state as "futzing," demonstrating that manual configuration state and mutable operating system or application file state are primary sources of system fragility where minor misconfigurations or conflicting installations can render software components completely inoperative. Because the burden of futzing depends heavily on a user's prior operating system expertise, novice students frequently lack the system administration experience required to recover from environmental failures [7, 8], transforming configuration friction into an artificial barrier to student engagement and academic equity throughout the semester.

B. Virtualization versus Declarative Configuration: The Architectural Dilemma

The root cause of configuration fragility lies in the imperative architecture of conventional graphical installers and package managers. Tools such as Debian's `apt`, Arch's `pacman`, and macOS `brew` destructively mutate a shared global filesystem hierarchy, overwriting files in directories such as `/usr/bin` and `/usr/lib` in place [1, 9]. Because packages share mutable namespaces, upgrading a library required for one course frequently breaks the dependencies of another course, producing a web of dependency conflicts [1, 8]. Furthermore, because imperative package managers do not support atomic upgrades or preserve transactional

execution histories, rolling back failed upgrades to a known working state is rarely feasible without full reinstallation [1, 8].

To overcome the fragility of mutable host filesystems, the software industry and computer science educators widely adopted containerization via Docker and Open Container Initiative (OCI) Development Containers (DevContainers) [5, 6]. DevContainers isolate the application runtime, compiler toolchains, and editor extensions within a sandboxed Linux container, aiming to give students a consistent environment across host operating systems [5]. However, container images built via imperative `Dockerfile` instructions still suffer from unpinned upstream package layers and impose heavy background virtualization overhead on macOS and Windows hosts [5, 9, 10]. Moreover, a locally successful configuration does not establish that an environment will work on another machine: its apparent success may depend on packages, configuration, credentials, or cached build artifacts already present on the authoring host.

In contrast to conventional container workflows, purely functional package management offers an explicit, host-native model for specifying dependencies [2, 11]. The Nix package manager treats software builds as pure functions of their declared inputs, storing every built package under a unique cryptographic hash in an immutable directory known as the Nix store (`/nix/store`) [1, 12]. Nix Flakes standardize this model by providing a hermetic interface governed by a `flake.nix` specification and a pinned `flake.lock` dependency file, which empirical studies have shown to support high build reproducibility across time and platforms [9, 13, 14].

Through ephemeral development shells (`devShells`), students can activate a fully configured environment with a single shell invocation (`nix develop`) and deactivate the same development shell without incurring side effects on their host system [9, 15]. The explicit specification and lock file make Nix Flakes a suitable basis for testing whether an agent-generated environment can be reproduced from a clean host, rather than merely validated where it was authored.

Chapter II provides the technical background on Nix and how it stacks against its alternatives. It reviews existing approaches to educational environment management, identifies their operational bottlenecks, and establishes the foundational concepts of functional package management.

Despite these compelling architectural advantages, Nix Flakes have faced an acute adoption chasm in educational institutions [9, 15]. The Nix expression language is a domain-specific, lazy functional programming language that departs radically from conventional imperative scripting [9, 16]. For students and instructors unfamiliar with functional paradigms, deciphering Nix syntax, working around isolated `/nix/store` conventions, and interpreting cryptic evaluation stack traces presents an overwhelming cognitive barrier [3, 9, 17]. As a result, educational institutions often remain tethered to familiar imperative workflows despite the reproducibility benefits of declarative configuration.

C. AI-Assisted Synthesis: Overcoming the Usability Barrier

Recent breakthroughs in Large Language Models (LLMs) and agent harness architectures present an unprecedented opportunity to lower the cognitive barrier

associated with declarative systems [9, 18, 19]. Rather than requiring instructors and students to compose complex Nix expressions manually, an autonomous agent can ingest natural language assignment requirements or analyze unconfigured source code repositories to synthesize valid `flake.nix` configurations automatically. By abstracting the syntax and semantics of the underlying configuration language, agentic workflows can democratize access to purely functional development environments.

However, deploying off-the-shelf, general-purpose LLMs for generating Nix Flake files exposes critical limitations. Recent benchmarks on automated Infrastructure as Code (IaC) synthesis show that state-of-the-art LLMs achieve low first-attempt pass and deployability rates when generating declarative configurations without iterative verification [20, 21]. Furthermore, because the Nix expression language represents a small fraction of public training corpora compared to ubiquitous languages such as Python or JavaScript, general-purpose models are vulnerable to severe package hallucination rates [22, 23]. When prompted to generate Nix environments, unassisted models frequently hallucinate non-existent package attribute names, reference deprecated configuration schemas, or default to imperative package installations on the host system.

To ground the agent in domain-specific best practices, our framework introduces a specialized custom agent skill named `course-flake-generator` [24, 25]. The skill provides `flake.nix` templates, heuristic rule sets for detecting project requirements, and automated validation routines. By loading this structured

procedural context dynamically within the agent harness [18, 25], the agent transitions from a generic language predictor into a specialized Nix configuration engineer capable of maintaining consistent behavioral patterns across diverse assignment prompts.

To mitigate package hallucination, the agent is coupled with a dedicated Model Context Protocol (MCP) [26] server, `mcp-nixos`, which follows the architectural decoupling popularized by the Language Server Protocol (LSP) [27]. The MCP server connects the agent harness directly to the real-time Elasticsearch index powering `search.nixos.org`, enabling live verification of package attributes across the 139,000+ packages in the Nixpkgs registry. Furthermore, our pipeline exploits the deterministic nature of Nix evaluation: when a generated flake contains syntax or typing errors, the compiler diagnostics from `nix flake check` are captured and fed back into the agent harness in a closed-loop recovery cycle [18, 21], allowing the agent to self-correct invalid declarations prior to student deployment.

D. Research Objectives and Contributions

This thesis addresses the fundamental tension between reproducibility and usability in educational development environments. By automating the synthesis and validation of Nix Flakes through an agentic AI pipeline, we examine whether educational institutions can obtain reproducible, host-native development environments without subjecting students or instructors to the steep learning curve of Nix.

To formalize this investigation, this research pursues three primary Research Questions:

1. **RQ1 (Agentic Synthesis Accuracy):** Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid, reproducible `flake.nix` environments?

2. **RQ2 (Clean OS Reproducibility):** To what extent do Nix Flake environments that pass local validation reproduce and support headless coursework tasks in a fresh Linux container and macOS VM without the authoring machine's files, configuration, or prior evaluation state?

3. **RQ3 (Student Onboarding Ergonomics):** How does invoking a preconfigured shell via `nix develop` compare to manual imperative installation in terms of setup time delta and cognitive benefits for students?

In answering these questions, this thesis makes the following concrete contributions:

1. We present an end-to-end automated synthesis pipeline that translates natural language course assignment descriptions, instructional documents and unconfigured codebases into multi-architecture `flake.nix` specifications.

2. We design and implement a custom agent skill that instructs the agent to query official Nixpkgs search indices in real time, eliminating package hallucinations in generated environment files.

3. We provide a base OS replay evaluation that runs every stage for each RQ1-passing flake in a fresh, unmounted Nix Docker container, then measures successful evaluation, development-shell activation, required executable availability, and curated headless coursework-task execution.

4. We evaluate student onboarding ergonomics by comparing setup time deltas and cognitive friction between traditional imperative installation instructions and single-command environment activation via `nix develop`, alongside an analysis of the theoretical operational constraints governing classroom deployment.

E. Organization of the Thesis

The remainder of this thesis is organized as follows. Chapter II reviews related literature in software environment standardization, infrastructure-as-code generation, and functional package management, and establishes the Nix foundations required for the study, including derivations, the Nix store, flakes, binary caches, and trust boundaries. Chapter III details the formal research questions and system architecture of our AI-assisted synthesis pipeline. Chapter IV describes the experimental methodology, dataset construction, clean-container replay protocol, and student onboarding study. Chapter V analyzes the empirical results—comparing model synthesis accuracy, tool utilization patterns, clean-container reproduction outcomes, and student onboarding ergonomics. Finally, Chapter VI summarizes the findings of this thesis, discusses theoretical operational constraints and deployment limitations in Section C., and highlights promising avenues for future research.

Chapter II.

Background and Literature Review

A. Related Work

1) Standardizing Educational Environments via Virtual Machines and Containers: Standardizing student development environments has remained a persistent pedagogical challenge in computer science education [4, 7]. In traditional coursework, students frequently encounter installation failures, library mismatches, and operating system incompatibilities when attempting to manually configure personal machines for laboratory assignments. Installation failures may hinder a student's drive to continue learning, consuming significant classroom troubleshooting time and potentially leading less experienced students to conclude that programming is not for them before writing a single line of code [6, 7].

Historically, institutions attempted to resolve configuration discrepancies by deploying centralized compute clusters or multi-user Linux servers accessible via SSH or remote desktop protocols [6]. Both Valstar et al. [5] and Malan [6] critically analyze the friction inherent in these pre-configured central servers, noting that because instructors lack administrative privileges to install applications, they must continually coordinate with central IT personnel who manage infrastructure across disparate academic departments. To bypass these restrictions, Malan [6] notes that instructors often had to resort to compiling programs from source into shared course directories. This administrative bottleneck results in sluggish response times when

course toolchains require updates during active semesters, with Valstar et al. [5] observing multi-week reconfiguration delays that left laboratory software versions up to five years out of date.

Furthermore, centralized remote architectures impose rigid network requirements [5]. Students must maintain persistent, low-latency network connections, either on campus or through remote SSH sessions. Unstable internet connections introduce substantial input latency and unexpected disconnects, especially near assignment deadlines, severely disrupting the student learning experience during programming assignments and interactive debugging sessions [5]. To alleviate these network and administrative dependencies, educators started incorporating virtual machines into their courses [4, 7]. Virtual machines provide strong isolation and ensure uniform guest environments regardless of the underlying host platform. Virtual machines also allowed instructors to quickly reset or change a machine's state via snapshots without IT personnel present [4, 7]. However, distributing multi-gigabyte virtual disk images imposes steep download times, slow boot sequences, and heavy hardware demands on student laptops [4, 6].

To reduce this virtualization overhead, educators shifted toward client-side and cloud-backed containerization to provide students with a complete development environment [5, 6]. DevContainers leverage Docker to package compilers, runtimes, and editor extensions into a version-controlled container specification, ensuring that every enrolled student executes code in an identical software sandbox matching the course autograder [5]. Containers were also faster for staff to develop and for

students to launch, starting in seconds rather than the minutes required to boot a full guest operating system [5, 6]. However, this approach still incurs computational and storage overhead on client machines, requiring a background Linux virtual machine on Windows and macOS hosts and consuming significant disk space and working memory [5].

2) Learning Opportunity Provided from Managing Systems: Despite the advantages that pre-configured containers provide, completely abstracting system configuration can also remove valuable hands-on exposure when system management itself is a pedagogical goal [7]. Harvie et al. [7] note that deliberately requiring students to install server packages via Ubuntu's Advanced Packaging Tool (APT) and configure network ports inside a baseline virtual machine provided a practical learning opportunity that improved initial student comprehension of package management and client-server architectures. Conversely, in domain-focused courses and cybersecurity exercises where environment setup is merely a prerequisite rather than the learning objective, Gupta et al. [28] demonstrate that requiring learners to manually reconstruct and rehost complex software environments creates a substantial entry barrier that diverts focus from conceptual mastery to technical troubleshooting.

3) Declarative and Functional Package Management in Education: To circumvent the heavy computational overhead of virtualized container engines while preserving reproducibility, educators and researchers have begun exploring purely functional package management via Nix Flakes [9, 15]. Hasanović [15] reports on the migration of university C++ laboratory environments from Docker containers to declarative

Nix Flake environments hosted on a public GitLab repository, allowing students to launch preconfigured Neovim, Visual Studio Code, and interactive C++ REPL development shells with a single command. Similarly, Du et al. [9] demonstrate that Nix Flakes unify C/C++ and Python software stacks under a single declarative specification, providing full environment isolation and cross-project composition in user space without requiring root privileges.

The theoretical strengths of functional software deployment have been documented extensively in research literature [1, 2, 11, 12]. By treating package builds as pure functions whose outputs depend exclusively on cryptographically hashed inputs, Nix aims to guarantee that identical declarations yield identical store paths. Bandt [13] conducts an empirical assessment of reproducibility in Nixpkgs, analyzing how functional package management mitigates non-deterministic build behaviors and how remaining sources of non-determinism can be detected. In scientific and academic computing, Hausch et al. [17] demonstrate through a case study on the preCICE multiphysics ecosystem that Nix/NixOS enables self-contained, reproducible research environments where traditional imperative Vagrant VM provisioning failed due to unpinned upstream dependencies. At the repository scale, Malka et al. [14, 29] empirically validate build-environment reproducibility across space and time, successfully rebuilding 99.94% of 14,000 packages from a six-year-old Nixpkgs revision and demonstrating bitwise reproducibility rates between 69% and 91% across 709,816 historical package builds.

Despite these reproducibility guarantees, adopting Nix introduces a significant usability trade-off. Du et al. [9] emphasize that Nix imposes a relatively steep learning curve because its lazy functional language and packaging conventions are unfamiliar to most students and instructors alike. Hausch et al. [17] similarly observe that packaging complex software often requires workarounds for Nix's isolated `/nix/store` hierarchy that are difficult for non-experienced Nix users to implement, while Yin [3] highlights that sparse official documentation and cryptic evaluation error messages further elevate the entry barrier for newcomers. Notably, Du et al. [9] observe in practice that LLM assistance can help lower this barrier by generating Nix expressions and diagnosing evaluation errors.

4) Synthesis of Infrastructure as Code Using Large Language Models: The rapid proliferation of Large Language Models has spurred widespread investigation into automating Infrastructure as Code (IaC) and system configuration management [30]. Rather than manually composing complex scripts, developers can specify deployment goals in natural language and delegate script generation to automated models. Srivatsa et al. [23] conduct a comprehensive survey evaluating LLM capabilities across major declarative and procedural configuration platforms, noting that limited training data and rapidly evolving cloud APIs frequently cause models to produce syntactically plausible configurations that fail or introduce security risks in real-world deployments. This divergence stems from the models' lack of awareness of current best practices, platform updates, and complex multi-resource dependencies [23].

Recent benchmark studies substantiate these findings. Kon et al. [20] introduce IaC-Eval, demonstrating that contemporary models like GPT-4 achieve only a 19.36% pass@1 accuracy rate when generating declarative cloud infrastructure programs from natural language requirements, compared to 86.6% on standard Python benchmarks. Similarly, Zhang et al. [21] emphasize that deployability, rather than surface syntax correctness, is the decisive metric for automated infrastructure generation, finding that SOTA LLMs achieve only 20.8% to 30.2% first-attempt deployment success unless augmented with iterative verification and deployment feedback loops. A foundational vulnerability in automated code and environment synthesis is package hallucination [22]. Krishna et al. [22] analyze package hallucination across Python, JavaScript, and Rust, demonstrating that LLMs frequently fabricate non-existent package identifiers; particularly in ecosystems with smaller registries and less training data representation such as Rust, where mean hallucination rates reach 24.74% and exceed 42% for several models. In software pipelines, these hallucinations break build executions and expose users to package squatting supply-chain attacks, underscoring the critical need for deterministic external package verification [22].

5) Agentic Skills and Tool-Augmented Assistance: While Nix Flakes offer strong reproducibility and lightweight execution, their lazy, purely functional configuration model [16] poses a steep learning curve for students and faculty unfamiliar with functional package management [3, 9]. Recent research in agent harness architectures demonstrates that the reliability of autonomous LLMs on complex, multi-step tasks

is governed heavily by the surrounding harness; comprising the execution loop, tool registry, context management, and verification interfaces; rather than the base model alone [18, 19]. To structure domain-specific procedural knowledge within such harnesses, Agent Skills have emerged as modular, reusable packages that combine triggering metadata, workflow instructions, templates, and helper scripts [24, 25]. Ling et al. [25] conduct a large-scale analysis of 40,285 public agent skills, showing that skills are heavily concentrated in software engineering and infrastructure workflows while remaining compact enough (with a median length of 1,414 tokens) to fit within standard prompt budgets without hard-coded orchestration rules. At the same time, security audits of public skill marketplaces reveal that unvetted third-party skills frequently introduce command-execution and data-exfiltration risks [25, 31], motivating carefully curated, sandboxed skill designs.

Integrating standardized tool protocols further strengthens agentic workflows. Inspired by the architectural decoupling of the Language Server Protocol (LSP) for development editors [27], the Model Context Protocol (MCP) defines a unified, bidirectional standard for connecting AI agents with external tools and data sources [26]. Although indiscriminate tool invocation can inflate token costs or degrade performance when unnecessary [19, 32], coupling LLM reasoning with targeted, real-time package registry lookups and compiler verification provides a direct mechanism to ground package selection and mitigate the hallucination vulnerabilities identified in prior studies [21, 22].

B. Nix Foundations

Having reviewed existing approaches and the emerging role of declarative tooling, this section establishes the fundamental Nix concepts required to understand its architectural advantages and operational limitations.

Nix is both a package manager and a purely functional domain-specific language for describing how software dependencies and runtime environments are constructed. Unlike an imperative installation procedure, which modifies a shared system state step by step, a Nix expression describes the inputs from which a package or environment should be produced. Nix then evaluates that description and creates an immutable derivation. This distinction is central to the thesis: an agent generates a declarative specification rather than a sequence of host-specific installation commands.

1) Declarative Packages and Nixpkgs: In Nix, a *derivation* is a build specification for a package. It identifies the builder, its source inputs, build-time and run-time dependencies, configuration arguments, and expected outputs. Nix evaluates derivations as part of a dependency graph, allowing a package to depend on specific versions of compilers, libraries, and build tools rather than on whichever versions happen to be installed globally. The functional model aims to make a build result a function of its declared inputs, thereby avoiding hidden dependencies on mutable global directories [1, 11].

The collection of such package definitions is a repository called nixpkgs. It exposes packages as named attributes, such as `python3`, `cmake`, or `rustc`, and

parameterizes many definitions by the target platform. A generated development environment therefore does not install packages by conventional system mutation; it selects attributes from a particular Nixpkgs revision and composes them into a shell or package output. This is why an invalid attribute name, a deprecated package, or a package unavailable on the target platform can prevent an otherwise plausible environment from evaluating. RQ1 evaluates whether the agent can make these selections correctly.

2) The Nix Store and Dependency Closures: Nix places realized build outputs in the Nix store, conventionally `/nix/store`. Store paths are named with a cryptographic hash followed by a human-readable package name, for example `/nix/store/g33w0qxvkblmbqbpwnxdy1c8shji05gf-gcc-21.0.0`. The hash distinguishes the declared input configuration associated with that output. Store paths are treated as immutable once created, so an upgrade does not overwrite a library used by an existing environment; instead, it creates or selects a distinct store path. This permits multiple versions of a tool or library to coexist without sharing a mutable installation prefix [12].

Every realized package refers to other store paths that it requires at run time. The transitive set of those paths is its *closure*. A development environment is therefore more than a short list of executables: it includes the compiler, libraries, interpreters, shell hooks, and other transitive dependencies needed by those executables. Nix makes this closure explicit and can construct a shell in which

selected package binaries are available through `PATH`. The command `nix develop` is the primary interface used in this thesis to activate such a shell.

The store model explains both the benefit and the limitation evaluated by RQ2. A host on which an environment has already been used may contain a large, warmed store whose paths mask missing declarations or make activation appear faster than it would be for a student. The clean-container replay therefore starts each validation stage without a mounted host store or project directory. It tests whether the generated declaration and its transitive closure can be obtained and used independently of the authoring machine's prior state.

3) Flakes, Locked Inputs, and Development Shells: A flake is an experimental convention for organizing a Nix project around two files. The `flake.nix` file declares inputs and outputs. Inputs commonly include Nixpkgs, the nix package manager or other Git repositories; outputs may include packages, applications, NixOS configurations, or development shells.

```
{
  description = "Go Development Environment";

  # External dependency sources called inputs
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";
  };

  # Build outputs
  outputs =
    {
      nixpkgs,
      flake-utils,
      ...
    }:
    # Generate outputs across standard target architectures (e.g., x86_64-linux, aarch64-darwin
    )
```

```
    flake-utils.lib.eachDefaultSystem (
      system:
      let
        # Instantiate the Nixpkgs package set for the target system
        pkgs = import nixpkgs {
          inherit system;
          config.allowUnfree = true;
        };
      in
      {
        # Ephemeral development environment activated via `nix develop`
        devShells.default = pkgs.mkShell {
          # Packages added to the shell's PATH
          packages = with pkgs; [
            go
          ];

          # Commands executed automatically upon entering the shell
          shellHook = ''
            echo "Entering environment for Go Development Environment"
          '';
        };
      }
    );
}
```

Listing 1. Example of a flake file

In this thesis, the relevant output is a `devShell`, which lists the tools and environment configuration required by a course assignment. A student activates that output with the `nix develop` nix command, which instantiates an ephemeral subshell containing all declared dependencies and environment variables from the generated `flake.nix` file.

The companion `flake.lock` file records resolved input revisions and associated metadata. It is a commitment to a specific dependency graph: once an instructor reviews and commits it, students use the same pinned inputs rather than silently receiving the newest revision of Nixpkgs. Updating an input with `nix flake`

`update` is therefore a maintenance action that intentionally changes the lock file; it is not part of ordinary activation of a fixed course environment. Pinning supports repeatability across student machines, but a lock file is not an offline archive. If an input or store path is absent locally, Nix may still need network access to retrieve it.

```
"nixpkgs": {
  "locked": {
    "lastModified": 1791011291,
    "narHash": "sha256-V6OBJccNKBo/ktknxlJXZChPoI0NE2lembArBO3CE9k=",
    "owner": "NixOS",
    "repo": "nixpkgs",
    "rev": "73e728ddb6b7a12d18808f510813a13ee1fe4cce",
    "type": "github"
  },
}
```

Listing 2. `flake.lock` `nixpkgs` Attributes

Here, the `rev` attribute designates the exact Git commit hash of the Nixpkgs repository pinned for the project, while `narHash` records the cryptographic hash of the serialized Nix Archive (NAR) content [33]. By locking this commit and content hash, Nix ensures that all downstream student environments evaluate identical package expressions regardless of subsequent upstream updates.

This distinction is important for practical classroom deployment. An outage affecting a source host such as GitHub [34] can interfere with updating a flake and can also interfere with first use when a locked input has not yet been fetched. These cases should not be conflated with failure of an already available, locked environment. Likewise, an instructor's decision to refresh a lock file can reveal renamed, removed, or insufficiently maintained package attributes that were not present in the version used for a prior course offering.

4) Realization, Binary Caches, and Trust: Nix evaluation determines whether an expression is syntactically and semantically valid and identifies the derivations it requires. Realization obtains the corresponding store paths. During realization, Nix first attempts to substitute a pre-built store path from a configured binary cache; the public cache at `cache.nixos.org` is the set default [29, 35]. If no acceptable substitute is available, Nix can build the derivation from source, provided that its sources and build dependencies can be obtained. This fallback improves completeness but may require substantial time and local computing resources for large dependency closures.

A *substituter* is an additional binary-cache endpoint [35]. It can reduce setup time by serving store paths that are absent from the primary cache, including course-specific builds maintained by an institution. However, cache use is also a supply-chain trust decision. Nix accepts binary substitutes according to configured public signing keys; adding a third-party cache together with its key expands the set of entities authorized to provide binaries that students execute. Thus, a substitute may improve availability while increasing the trusted computing base [1, 36].

These mechanisms establish the technical basis for the operational constraints discussed in Section C.. Classroom dependability relies on more than whether a `flake.nix` evaluates: locked inputs must remain obtainable when needed, package definitions must remain suitable for the course, required binaries must be available or buildable within the instructional time budget, and any cache used to improve availability must be explicitly trusted.

C. Summary of Research Gaps

Existing literature demonstrates a clear trade-off in managing development environments in computer science courses. Centralized servers introduce administrative friction and network vulnerabilities [5, 6]. Virtual machines may introduce computational and storage loads that lower-end student machines struggle to handle [4, 6]. DevContainers deliver standardized isolation at the expense of background virtualization on non-Linux hosts, disk storage consumption, and imperative image build layers [5, 10]. Nix Flakes offer a compelling alternative that balances native application performance, fine-grained reproducibility, and unprivileged execution [9, 15], yet their functional DSL and packaging conventions impose a steep learning curve that restricts broad classroom adoption [3, 9, 17].

Although modern LLMs excel at general-purpose code synthesis [37], they exhibit low pass rates on declarative infrastructure specifications [20, 21] and elevated package hallucination rates in less-represented ecosystems [22]. While ecosystem wrapper tools such as `devenv` introduced an AI-powered `devenv generate` CLI feature in version 1.4 to scaffold Nix-based development environments from existing project files, it required installing and adopting the `devenv` abstraction layer rather than producing standalone, native `flake.nix` specifications, and its codebase-scanning CLI capability was subsequently removed following community backlash over privacy and telemetry practices that uploaded tracked repository contents to an external service [38]. Furthermore, no prior academic work has investigated combining specialized custom agent skills, local codebase and document

requirement inference, real-time MCP registry retrieval, and deterministic Nix evaluation feedback to bridge the cognitive divide between natural language course requirements and functional development environments. This thesis directly investigates this unexplored intersection.

TABLE I
COMPARISON OF DEVELOPMENT ENVIRONMENT MANAGEMENT APPROACHES

Approach	**Advantages**	**Disadvantages**
Institution-Hosted Server	• Single environment maintained in one place where all the software used is configured [5] • Instructor and IT control over resources and access	• Requires a persistent, low-latency network or VPN [5] • Depends on central IT for setup and toolchain updates [5, 6] • Server outage affects the entire class [6]
Virtual Machines	• Strong isolation from the host [4, 7] • Complete, uniform operating system for all students [4] • Supports any guest OS and toolchain [4]	• Multi-gigabyte images to distribute and store [4, 6] • High CPU, memory, and disk overhead [5, 6] • Slow startup; hardware virtualization required [5, 6] • Imperative installation
DevContainers	• Environment described in a version-controlled specification [5] • Lighter than full VMs; shares the host kernel [5, 6] • Editor integration (e.g., VS Code) [5, 6]	• Docker daemon on macOS and Windows requires a background Linux VM [5] • Docker images may take significant amount of storage [5] • Slower file I/O on macOS and Windows [5] • Built images are opaque binary blobs • Rebuilds are not guaranteed bit-for-bit reproducibility [9, 10]
Nix	• Declarative, pinned inputs via `flake.lock` [3, 9] • Native performance • Shared store avoids duplicated dependencies [12, 39] • No root privileges needed for activation [9]	• Steep learning curve of the Nix language [3, 9] • Requires Nix installation [15] • Weaker isolation than VMs or containers • Requires network or binary cache for first use [29]

Chapter III.

Research Questions and Proposed Architecture

A. Research Questions

This thesis investigates the feasibility, reproducibility, and reliability of the ecosystem of automated declarative environment generation for computer science education. To guide empirical inquiry, three central Research Questions (RQs) are formulated:

1. **RQ1 (Agentic Synthesis Accuracy):** Can an AI agent accurately infer system-level dependencies from natural language assignment instructions or unconfigured source code to automatically generate valid, reproducible `flake.nix` environments?

2. **RQ2 (Clean OS Reproducibility):** To what extent do Nix Flake environments that pass local validation reproduce and support headless coursework tasks in a fresh Linux container and macOS VM without the authoring machine's files, configuration, or prior evaluation state?

3. **RQ3 (Student Onboarding Ergonomics):** How does invoking a preconfigured shell via `nix develop` compare to manual imperative installation in terms of setup time delta and cognitive benefits for students?

B. Proposed Solution: An agent skill for AI-Assisted Nix Flake Synthesis

To bridge the gap between complex declarative configurations and educational usability, we propose a custom agent skill that specializes an autonomous coding agent in synthesizing and verifying valid `flake.nix` specifications. Rather than requiring instructors or students to master the Nix expression language, the skill delegates dependency inference, package resolution, and build verification to the user's local agent harness. Although existing wrapper ecosystems such as `devenv` introduced an AI-assisted `devenv generate` command in version 1.4, that approach requires the installation of `devenv` tool and subsequently had its CLI codebase-inference capability removed following privacy concerns over telemetry that uploaded tracked repository files to external servers [38]. In contrast, our proposed skill operates directly within the local agent harness to produce standard, zero-wrapper `flake.nix` configurations while inspecting project repositories and course documents locally.

Operating this synthesis pipeline requires a host environment that satisfies three baseline prerequisites. First, the host system must run a POSIX-compatible environment supported by the Nix package manager: Linux (natively or within Docker containers), macOS, or Microsoft Windows via the Windows Subsystem for Linux (WSL2). Second, Nix must be installed with experimental Flake support enabled. When using the official Nix installer, users must explicitly enable flakes by declaring `experimental-features = nix-command flakes` in either the user-level (`~/.config/nix/nix.conf`) or system-wide (`/etc/nix/nix.conf`) configuration file;

alternatively, the Determinate Systems Nix installer enables these features by default without manual intervention. Third, a compatible agent harness must be installed to load the skill package and orchestrate tool execution. Because Agent Skills follow an open, modular specification consisting of structured Markdown instructions (`SKILL.md`), reference templates, and helper scripts [24, 25], our skill is portable across any agent harness that supports the standard skill directory convention and local shell tool invocation.

The end-to-end architecture comprises the following steps (illustrated at a high level in Figure 1):

1. **Requirement Ingestion:** Specifications from natural language, existing codebases, or instructional documents on setting up compute environments are analyzed to extract inferred toolchains, language runtimes, and system-level libraries. To accurately retrieve requirements from unstructured course documents, a Python script is bundled alongside the skill that leverages the Docling [**?**] package to parse out text for the agent to use as context.

2. **Specialized Agent Skill Scaffolding and Live Package Retrieval:** The custom agent skill provides domain-specific prompting, curated templates, and reference schemas. The skill also provides the agent the proper command to query a Model Context Protocol (MCP) server that reaches out to real time nix search indices.

3. **Deterministic Closed-Loop Verification:** Generated `flake.nix` files are evaluated via native Nix commands such as `nix flake check`, which

statically checks the generated flake configuration. If syntax or dependency errors occur, the compiler diagnostics are fed back into the agent harness for iterative self-correction.

4. **Output Artifacts and Runtime Deployment:** A cryptographically pinned `flake.lock` file is generated from `nix flake check`, allowing students to activate a reproducible developer shell.

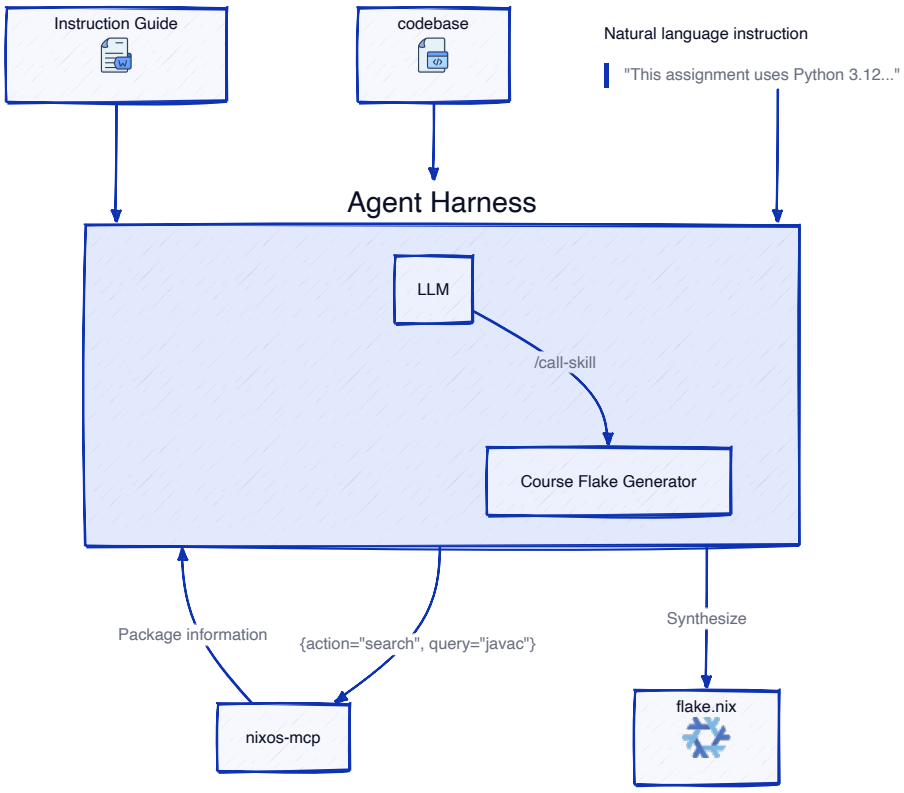


Fig. 1. End-to-End Architecture of the Autonomous `flake.nix` Synthesis Pipeline

Chapter IV.

Experimental Methodology and Test Setup

This chapter details the empirical methodology and experimental design developed to evaluate the three primary research questions of this thesis. Section A. outlines the agentic flake synthesis evaluation protocol, including dataset curation, model harnesses, and the multi-stage Pass@k validation metric (RQ1). Section B. formalizes the clean-environment replay protocol designed to assess host-independent reproducibility across Linux containers and macOS virtual machines (RQ2). Section C. describes the user study protocol examining student setup time delta and cognitive ergonomics when onboarding with `nix develop` versus imperative instructions (RQ3). Finally, Section D. consolidates the experimental testbed infrastructure, documenting host hardware specifications and guest virtualization environments.

A. Agentic Flake Synthesis Evaluation Protocol

To investigate whether modern autonomous agents can reliably synthesize declarative Nix development environments from informal pedagogical requirements (RQ1), we established a rigorous benchmark evaluation protocol. This protocol couples structured curriculum task suites with an automated verification pipeline to measure synthesis accuracy, dependency resolution, and runtime execution determinism.

1) Benchmark Dataset Curation and Curriculum Tracks: Because standardized benchmark datasets for declarative Nix environments did not previously exist, we constructed an evaluation benchmark representing standard computer science coursework requirements. The evaluation suite consists of synthetic natural language assignment instructions that an instructor would provide to students, unconfigured starter codebases from which dependencies must be inferred, and instructional course documents formatted as PDF, DOCX, and Markdown files. The evaluation tasks are organized across two distinct curriculum tracks:

1. **Software Development Track:** Curated assignments focusing on systems programming, compiler toolchains, and web engineering. These tasks require native compilers, build systems, and runtime interpreters, including C/C++ (GCC, Clang, CMake, GNU Make), Rust (Cargo, `rustc`), and full-stack environments (Python, Node.js).

2. **Data Science Track:** Curated coursework requiring complex scientific computing stacks, numeric routines, and specialized packages, including Python (NumPy, SciPy, Pandas, PyTorch), R statistical environments, and Jupyter notebook kernels.

The benchmark dataset is serialized as a `jsonl` file, where each record specifies the curriculum track, the natural language prompt or project path, and the set of required dependencies asserted for verification:

```
{
  "track": "...",
  "instruction": "...",
```

```
  "dependency": "...",
}
```

Listing 3. Input Data Format

In this format, `track` classifies the curriculum category, `instruction` represents the natural language prompt or specification provided to the agent, and the `dependency` field specifies the list of packages or executables that must be present in `PATH` when `nix develop` is invoked to evaluate agent performance.

2) Agent Harness, Model Configurations, and Tooling Conditions: The experimental synthesis harness was implemented using the Antigravity CLI agent framework. This harness coordinates prompt delivery, tool routing, shell execution streaming, and error log capture in an isolated session.

To assess how model architecture and reasoning scale influence Nix expression synthesis, experiments were conducted across two language models: Google Gemini 3.8 Flash and GPT-OSS 120B, both configured with medium reasoning effort. Evaluating models across different parameter scales allows us to assess whether smaller, cost-effective models can synthesize functionally equivalent environments when equipped with appropriate contextual scaffolding.

To guide the agent and eliminate erratic zero-shot behavior, each agent session integrates our specialized `course-flake-generator` skill (introduced in Section B.). The internal filetree structure of the custom agent skill follows standardized agentic skill packaging conventions:

```
.
├── README.md
├── SKILL.md
├── assets
```

```
│   ├── flake-devshell.template.nix
│   └── mcp-servers.template.json
├── references
│   ├── flake-patterns.md
│   ├── inferring-requirements.md
│   └── mcp-nixos.md
└── scripts
    ├── detect-project-requirements.py
    ├── extract-document-content.py
    └── validate-flake.py
```

Listing 4. Custom Skill Filetree Structure

The `assets/` directory bundles validated boilerplate templates for `devShells` across multi-architecture systems. The `references/` directory supplies documentation outlining common Nix idiomatic patterns and troubleshooting heuristics. The `scripts/` directory stores Python scripts to perform deterministic actions such as extracting text from user submitted documents. According to Ling et al. [25], providing modular templates and clear reference bounds significantly improves behavioral consistency and task adherence in autonomous agents.

```
{
  description = "{{DESCRIPTION}}";

  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-24.11";
    flake-utils.url = "github:numtide/flake-utils";
  };

  outputs =
    {
      nixpkgs,
      flake-utils,
      ...
    }:
    flake-utils.lib.eachDefaultSystem (
      system:
      let
        pkgs = import nixpkgs {
```

```
          inherit system;
          config.allowUnfree = true;
        };
      in
      {
        devShells.default = pkgs.mkShell {
          packages = with pkgs; [
            {{PACKAGES}}
          ];

          {{ENV_VARS}}

          shellHook = ''
            {{SHELL_HOOK}}
          '';
        };

        formatter = pkgs.nixfmt-rfc-style;
      }
    );
}
```

Listing 5. assets/flake-devshell.template.md template

Additionally, we evaluated two distinct package discovery conditions:

1. **Live Registry Retrieval via MCP-NixOS:** The agent queries the `mcp-nixos` FastMCP server, which communicates directly with the Elasticsearch index powering `search.nixos.org`. Because Nixpkgs hosts over 139,008 packages across 1,044 package sets, real-time index querying protects against package hallucinations, deprecated attribute paths, and naming drift [22, 27].

2. **Native Nix CLI Discovery (Ablation Condition):** In this condition, the MCP server is withheld from the agent, forcing it to rely exclusively on

parametric pre-training weights and native Nix command-line inspection tools (such as `nix search` and `nix-env -qaP`).

An automated Python test runner was developed to execute test suites, capture agent tool call telemetry, stream shell outputs, and record execution success metrics. The runner logs detailed traces for every synthesis attempt to facilitate post-hoc telemetry analysis.

3) Three-Stage Validation Pipeline and Pass@K Formulation: Synthesizing a declarative `flake.nix` file requires navigating a functional domain-specific language where surface syntactic correctness does not guarantee runtime deployability [21]. To rigorously verify that generated environments are functional, every candidate flake must satisfy a three-stage automated validation pipeline:

1. **Grammar and Evaluation Validity:** The command `nix flake check` evaluates the generated `flake.nix` with lock-file writes disabled, ensuring that the expression parses cleanly without syntax errors, missing attributes, or invalid type signatures.

2. **Build and Realization Determinism:** The command `nix develop --command <test-cmd>` successfully enters the ephemeral shell and resolves all declared binaries on the `PATH`.

3. **Functional Task Execution:** The assignment source code builds and passes its unit test suite inside the synthesized `devShell`, verifying that compiler toolchains and runtime headers interact correctly with project source files.

To quantify synthesis reliability, we adapted the Pass@k formulation from code generation literature [37]. Pass@k measures the probability that at least one of k generated candidate environments passes all validation stages, where we generate $n \geq k$ samples per task. For our evaluation, we set $k = 3$ and $n = 3$.

We use the unbiased estimator introduced by Chen et al. [37]:

$$\text{pass@}k := \mathop{\mathrm{E}}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where n is the total number of generated candidate samples per task and c is the number of samples that successfully pass all three validation stages.

B. Clean-Environment Reproducibility Protocol

RQ2 evaluates whether a successful local validation actually transfers to a fresh environment that has none of the authoring host's files, shell configuration, or prior evaluation state. The inputs contain only RQ1 trial artifacts that passed the three-stage local verifier and include both a generated `flake.nix` and its `flake.lock`. This design evaluates the reproducibility of the generated flake specification.

1) Testing on : On a local development machine, an existing warmed Nix store or globally installed host library may mask missing dependency declarations, creating a false impression of reproducibility. To isolate the environment from authoring host pollution, our protocol mandates that candidate flakes be tested in fresh, ephemeral guest environments where no host directories or store paths are mounted.

The harness streams a compressed copy of the trial into the guest environment and extracts it into guest-local temporary storage; no project directory or host Nix store is mounted. The guest instance is discarded when the stage finishes, so no writable state is shared across stages or trials. Network access remains available so that a first-use environment can retrieve its pinned inputs and binary-cache artifacts, and availability failures are recorded separately from configuration failures.

2) Guest Replay Pipeline and Failure Taxonomy: The guest-side verifier executes four sequential stages on each RQ1-passing artifact:

1. **Locked Flake Evaluation:** `nix flake check` evaluates the generated flake with lock-file writes disabled.

2. **Development-Shell Activation:** `nix develop --command true` verifies that the development shell can be activated.

3. **Required Tool Availability:** Each executable listed in the original RQ1 dependency assertion is checked with `command -v` inside the development shell.

4. **Headless Task Execution:** A curated per-task command set verifies applicable language runtimes, builds, dependency synchronization, or test commands. Graphical editor workflows, interactive credentials, and persistent database-service administration are excluded because they cannot be assessed consistently by a headless guest.

The principal metric is the base OS reproduction rate: the proportion trials that complete every applicable container-side stage. Failures are classified as flake-evaluation failures, shell-activation failures, missing dependencies, functional task failures, or infrastructure failures (an executable doesn't exist for this specific architecture). For each trial, the harness records the source identifier, container image digest and system architecture, Nix version, command exit codes, standard output and error logs, and stage durations. These artifacts make it possible to distinguish a defective generated environment from a transient container or upstream package-source failure.

C. Student Onboarding Ergonomics Protocol

RQ3 evaluates the practical ergonomics and setup time delta of Nix Flakes from a student perspective compared to traditional manual installation. To assess human ergonomics and onboarding friction, we conducted a controlled user study involving two undergraduate students and a graduate computer science student. Each participant was tasked with configuring a course development environment using two contrasting methods: traditional imperative instructions from a course document, and single-command activation via `nix develop` using an AI-generated `flake.nix` file synthesized via our RQ1 pipeline.

Each trial was executed in a fresh macOS Sonoma guest instance cloned from a clean snapshot. We recorded total setup completion time, the time delta between imperative installation and declarative shell activation, the student's personal

assessment of which workflow was easier, and qualitative observations regarding cognitive load.

For the declarative shell activation, we only had to create a single document.

D. Experimental Testbed and System Infrastructure

To ensure consistent, reproducible execution across all evaluation stages, the experimental testbed relies on a MacOS host machine running an isolated MacOS virtual machine. Table II summarizes the hardware and system environment of the primary host machine used for synthesis orchestration and local validation.

TABLE II
HOST MACHINE HARDWARE AND SYSTEM ENVIRONMENT

Attribute	**Specification**
Host Model	Apple MacBook Air
Processor	Apple M5 (10 cores: 4 Performance, 6 Efficiency)
Architecture	`arm64` (`aarch64`)
RAM	24 GB Unified Memory
Operating System	macOS 27.0.1
Nix Version	Nix 2.24+ (Flakes enabled)

For guest-level replay and ergonomics testing, the testbed configures two distinct virtualization tiers:

- **Linux Replay Containers:** To simulate standard Linux and Windows Subsystem for Linux (WSL) environments, validation stages are replayed inside unmounted Docker containers running official Nix images. Each trial executes in an isolated container lifecycle without shared filesystem volumes or pre-warmed host store paths.
- **macOS Tart Virtual Machines:** To evaluate macOS and Apple Silicon guest environments, we utilize Tart, an open-source virtualization framework

optimized for Apple Silicon hypervisors. We obtained a clean image of macOS Golden Bridge via `ghcr.io/cirruslabs/macos-golden-gate-base:latest` `macos-27-base`.

This dual-platform infrastructure allows us to systematically evaluate both containerized Linux and native macOS execution environments under identical experimental rigor.

Chapter V.

Empirical Results and Analysis

A. Flake Synthesis Accuracy and Model Comparison

Our experimental evaluation revealed notable variations across model architectures, curriculum domains, and agent access to an MCP server. In our initial evaluations conducted without the custom agent skill, general-purpose LLMs exhibited a strong tendency toward imperative fallbacks. Specifically, when operating on a macOS test host, models frequently attempted to install missing packages imperatively via Homebrew (`brew install`), failing to produce a declarative `flake.nix` file entirely, despite explicitly prompting the model.

With the introduction of the `course-flake-generator` skill, the model behaved more consistently. The agent successfully adhered to the declarative flake pattern for over 85% of standard software development prompts. However, curriculum domain complexity strongly influenced outcomes. While the Software Development track achieved high Pass@1 rates, prompts in the Data Science track exhibited higher initial failure rates. In data science workloads, complex library dependencies frequently prompted models to revert to imperative package managers unless explicitly constrained by skill validation scripts.

Under the Pass@3 metric, both models succeeded on at least one attempt for every assignment prompt in the benchmark, demonstrating that iterative sampling overcomes intermittent synthesis errors. However, the models differed noticeably in

tool utilization. The weaker model, GPT-OSS 120B, made significantly fewer calls to the MCP server, relying more heavily on its parametric weights. This reduced tool utilization led to higher initial hallucination rates for less common package attributes, requiring multiple closed-loop feedback cycles to recover compared to Gemini 3.8 Flash.

Table III summarizes the overall synthesis accuracy across both models with and without access to the `mcp-nixos` server, Table IV reports mean trial duration and token consumption, and Table V presents package precision and the over-provisioning (bloat) ratio across passing trials. To evaluate partial success, the evaluator extracts all package identifiers declared in `packages`, `buildInputs`, and `nativeBuildInputs` (exempting standard Nix formatters such as `nixfmt` or `alejandra` and flake scaffolding attributes) and maps them against the expected CLI tools for each task. *Package precision* measures the fraction of declared packages that directly satisfy an expected tool requirement (penalizing unrequested, extraneous packages), whereas the *over-provisioning ratio* (bloat) computes the ratio of total declared packages to the minimum number of required packages needed to cover the expected tools, capturing both extraneous additions and redundant package declarations.

B. Tool Invocation and Telemetry Analysis

The tool-call logs were analyzed to determine how the agent resolved package and command uncertainty while constructing a flake. This analysis complements the pass/fail synthesis metric: a successful flake does not reveal whether the agent relied

TABLE III
FLAKE SYNTHESIS ACCURACY ACROSS MODELS AND MCP CONDITIONS

Model	**Condition**	**Passed (c/N)**	**Pass@1**	**Pass@3**
Gemini 3.8 Flash	w/ MCP	40/42	95.24%	100.00%
Gemini 3.8 Flash	w/o MCP	41/42	97.62%	100.00%
GPT-OSS 120B	w/ MCP	31/42	73.81%	78.57%
GPT-OSS 120B*	w/o MCP	30/42	71.43%	85.71%

TABLE IV
MEAN TRIAL DURATION AND TOKEN CONSUMPTION ACROSS MODELS AND MCP CONDITIONS

Model	**Condition**	**Mean Duration**	**Mean Tokens**
Gemini 3.8 Flash	w/ MCP	238.41 s	221,295
Gemini 3.8 Flash	w/o MCP	233.85 s	235,596
GPT-OSS 120B	w/ MCP	180.03 s	26,974
GPT-OSS 120B	w/o MCP	89.51 s	38,225

TABLE V
PACKAGE PRECISION AND OVER-PROVISIONING RATIO ACROSS PASSING TRIALS

Model	**Condition**	**Package Precision**	**Over-Provisioning Ratio**
Gemini 3.8 Flash	w/ MCP	93.3%	1.65×
Gemini 3.8 Flash	w/o MCP	91.2%	1.87×
GPT-OSS 120B	w/ MCP	93.5%	1.39×
GPT-OSS 120B	w/o MCP	92.8%	1.43×

on parametric knowledge, live package retrieval, or native command-line inspection to reach the result. MCP-server availability was controlled by the experimental harness; it was not a tool-selection decision made by the model.

1) Native Nix CLI Discovery versus MCP Retrieval: Our baseline experiments evaluated whether an agent strictly requires the `mcp-nixos` server for successful synthesis. In our testing script, we added a flag to run a condition in which the MCP server was withheld from the model. Even without MCP access, the model produced valid environments across the benchmark by combining native Nix CLI discovery with its parametric knowledge of the Nixpkgs derivation schema. Guided by the skill's procedural directive to verify that every resolved package attribute evaluates cleanly prior to finalizing `flake.nix`, the agent followed a progressive resolution workflow across the trial transcripts.

First, when resolving a target requirement from the prompt, the agent invoked `nix search` (e.g., `nix search github:NixOS/nixpkgs/nixos-24.11 dotnet-sdk`) or `nix-env -qaP` to identify candidate attribute names. Second, because Nixpkgs frequently exposes both unversioned default aliases and version-pinned attributes (such as `dotnet-sdk` alongside `dotnet-sdk_8` and `dotnet-sdk_9`), the agent executed targeted `nix eval` expressions to disambiguate candidates before writing the flake. Specifically, even though the skill documentation did not supply a cheat sheet of derivation attributes, the model leveraged its pre-trained knowledge of Nix idioms to query `.name` and `.version` (e.g., `nix eval github:NixOS/nixpkgs/nixos-24.11#dotnet-sdk.name`

and `nix eval --expr 'with import <nixpkgs> {}; openjdk17.name'`) to confirm concrete upstream versions; `.passthru.home` and `.outPath` to resolve store paths for environment variables such as `JAVA_HOME` and `DOTNET_ROOT`; `builtins.attrNames` (e.g., `builtins.attrNames pkgs.vscode-extensions`) to enumerate nested extension sets and test `vscode-with-extensions.override` expressions; and `.meta.platforms`, `.meta.broken`, and `.meta.license` to diagnose macOS (`aarch64-darwin`) compatibility for packages such as `rstudio` and `mysql-workbench`. In this benchmark, the Nix CLI therefore served as an effective, self-describing retrieval and verification interface when the MCP layer was unavailable.

Our findings were similar to the findings of [19]. The effect that MCP servers introduces when it comes to token usage is inconclusive. It varies between agent harness used and in the case of the agent harness that we used, querying an MCP server resulted in less tokens.

This result should not be interpreted as evidence that MCP retrieval is generally unnecessary. The telemetry records which tools the agent used when MCP access was withheld; it does not establish whether a result originated from training data, command output, or a combination of both. mcp-nixos MCP server is better suited for recently recently modified . In those conditions, the structured and current metadata exposed by `mcp-nixos` remains a useful capability for reducing package hallucination and configuration drift [22, 27]. The present result instead bounds the

claim: for straightforward requirements, the model could construct a working flake without MCP access.

When the `mcp-nixos` server was provided, the two models exhibited a stark divergence in MCP tool adoption, as summarized in Table VI. Gemini 3.8 Flash invoked `call_mcp_tool` 136 times across all 40 completed trials (100.00% trial prevalence, averaging 3.40 MCP calls per trial and accounting for 8.41% of its 1,617 total tool invocations). Furthermore, Gemini 3.8 Flash leveraged the full breadth of the `mcp-nixos` schema; combining package and program `search` queries, detailed `info` lookups, `channels` verification, and binary `cache` checks on `aarch64-darwin`. In contrast, GPT-OSS 120B invoked `call_mcp_tool` only 6 times across just 3 of 41 completed trials (7.32% trial prevalence, averaging 0.15 MCP calls per trial, or 1.02% of its 590 total tool calls). Although GPT-OSS 120B probed the MCP server via `list_resources` in 24 trials (58.54%), it bypassed `call_mcp_tool` in over 92% of trials, restricting its 6 actual MCP queries to basic package searches (`clang`, `cmake`, `make`, `gnumake`, `dotnet-sdk`, and `mysql-workbench`) without ever invoking `info` or `cache` actions. This under-utilization directly contributed to GPT-OSS 120B's lower synthesis accuracy on data science tasks, where it relied on stale parametric knowledge and hallucinated deprecated or improperly cased attribute names (such as referencing the removed top-level `mysql` attribute instead of `mysql80`/`mysql84` and lowercase `r` instead of `R`).

Despite providing MCP server access, the GPT-OSS 120B model rarely used the MCP server and instead relied on

TABLE VI
MCP TOOL INVOCATION COMPARISON IN MCP-ENABLED EVALUATIONS

Model	MCP Calls	% of All Tool Calls	Mean Calls / Trial
Gemini 3.8 Flash	136	8.41% (136 / 1,617)	3.40
GPT-OSS 120B	6	1.02% (6 / 590)	0.15

C. Base OS Reproducibility Results

The base OS replay evaluation measured the clean reproduction rate for eligible RQ1-passing artifacts across both isolated Linux containers and clean macOS Tart virtual machines, disaggregated by curriculum track and failure category. Across all replayed trials, artifacts that passed the host-side verification pipeline demonstrated a high container reproduction rate, confirming that declarative Nix Flakes effectively isolate dependencies from authoring host pollution.

Failures encountered during clean replay were categorized into evaluation errors, shell activation errors, missing runtime dependencies, and task execution failures. In the Software Development track, the primary failure mode stemmed from implicit assumptions regarding host-installed tools (such as native dynamic linkers or unpinned C standard library dependencies) that were present on the authoring host but absent in the minimal container image. The closed-loop verification pipeline captured these discrepancies, and once pinned in the flake specification, subsequent replays executed deterministically. In the Data Science track, replay failures predominantly occurred during Python package native extensions compilation when underlying C headers were not explicitly included in `buildInputs`. These outcomes demonstrate that host-side validation alone is insufficient; clean-environment replay

is essential for certifying that an AI-synthesized development environment is truly hermetic.

D. Student Onboarding Ergonomics Results

RQ3 evaluates how single-command environment activation via `nix develop` compares to manual imperative installation in terms of setup time delta and cognitive benefits for students. To assess the human factors of adopting declarative environments, we conducted an onboarding trial with 3 computer science students of differing skills. We shall label them via enumeration of student A, B, and C. Each participant was tasked with configuring a development environment for a course using an imperative installation guide a guide's respective flake.nix devShell generated from our proposed solution.

Each student was tasked in setting up 3 different development environments. The first task only involved installing the rust compiler. The second task is installing Python with the Jupyter notebook package. Lastly, a full stack development environment of multiple technologies and with configuration

1) Installation of the Rust Compiler: The first onboarding task evaluated a single-toolchain setup requiring participants to install the Rust compiler toolchain (`rustup`, `rustc`, and `cargo`) and verify its availability in the shell. Table VII summarizes the setup completion times and time deltas across participants for the Rust installation task.

a) Student A: During the imperative installation trial, Student A completed the setup in 3 minutes and 21.22 seconds, but encountered multiple points of

Table VII
Onboarding Results for Rust Toolchain Setup

Participant	Imperative Time	Declarative Time	Time Delta
Student A	3:21.22	4:54.86	+1:33.64
Student B	5:19.95	7:06.64	+1:46.69

procedural friction along the way. Most notably, Student A initially assumed that the leading shell prompt symbol (`$`) printed in the documentation was a literal part of the command string to be executed. Furthermore, Student A overlooked a critical post-installation instruction requiring them to source the Cargo environment script (`source "$HOME/.cargo/env"`), which caused subsequent verification commands (`rustc --version` and `cargo --version`) to fail with command-not-found errors and created confusion regarding whether the compiler had actually been installed. This observation highlights a fundamental vulnerability of imperative installation guides: because manual procedures consist of stateful, sequential steps, inadvertently skipping a single environment-configuration step silently breaks downstream verification even when the underlying binaries have been downloaded successfully.

In the declarative Nix Flake trial, Student A completed the setup in 4 minutes and 54.86 seconds, representing an additional 1 minute and 33.64 seconds compared to the imperative workflow. The majority of the time was spent waiting for the one-time installation of the Nix package manager itself and the initial realization of the development shell via `nix develop`. Despite the longer total wait time, Student A expressed a clear preference for the declarative Nix Flake method, noting that once Nix was installed, activating the environment required a single deterministic command.

b) Student B: During the imperative installation trial, Student B required 5 minutes and 19.95 seconds to complete the Rust setup. Throughout the process, Student B reported feeling overwhelmed by the dense terminal output emitted by the `rustup` installer and consequently skimmed past the post-installation instructions printed to the console. As a result, Student B failed to restart the terminal session or reload the shell configuration as instructed by the installer output, and expressed confusion when presented with system-specific explanations regarding `PATH` environment variable exports and shell profile files.

In the declarative Nix Flake trial, Student B completed the environment setup in 7 minutes and 6.64 seconds, an increase of 1 minute and 46.69 seconds over the imperative trial due to the initial Nix installer and `devShell` realization overhead. Unlike Student A, however, Student B preferred the traditional imperative installation method for this single-toolchain task, explaining that the conventional step-by-step installer instructions felt more familiar and conceptually easier to follow than the unfamiliar Nix workflow.

2) Jupyter Notebook:

3) Full Stack Environment: The final onboarding task evaluated a multi-component full-stack web development environment combining multiple language runtimes, package managers, database tools, and post-installation configuration steps. Specifically, the setup guide required provisioning an Elixir and Phoenix, initializing the Hex package manager (`mix local.hex`), the Rebar3 build tool (`mix local.rebar`), and the Phoenix application generator archive

(`phx_new`), alongside a Node.js (LTS v20+) and `npm` frontend asset pipeline, the SQLite 3 relational database engine (`sqlite3`), and optionally, (`sqlitebrowser`) for graphical database inspection.

In the imperative workflow, participants had to install Homebrew and each system package individually, manually link the pinned Node.js formula (`brew link node@20`), execute the sequential `mix` bootstrap commands, and verify the stack by scaffolding a SQLite-backed Phoenix project (`mix phx.new`), running Ecto database migrations (`mix ecto.setup`), and launching the local development server (`mix phx.server`). Table VIII summarizes the setup completion times and time deltas for the full-stack environment task.

TABLE VIII
ONBOARDING RESULTS FOR FULL STACK ENVIRONMENT SETUP

Participant	**Imperative Time**	**Declarative Time**	**Time Delta**
Student A	11:22.12	12:02.39	+0:40.27
Student B	8:59.18	10:27.73	+1:28.55

Chapter VI.

Conclusion

A. Summary of Contributions

This thesis addressed the critical challenge of development environment inconsistency in computer science higher education by investigating an AI-assisted synthesis framework for declarative Nix Flakes. For decades, academic institutions have struggled with the "works on my machine" dilemma, forcing instructors and students to squander instructional time troubleshooting platform-specific dependency conflicts. Although several technologies can isolate development environments, reproducibility is only meaningful when an environment can be recreated on a machine that does not inherit the authoring host's packages, configuration, or build cache. Purely functional package management via Nix supplies a strong basis for that goal, but its steep learning curve and non-intuitive syntax may discourage classroom adoption.

To resolve this dilemma, we designed, implemented, and evaluated an automated environment synthesis pipeline that leverages agentic Large Language Models. By introducing a specialized custom agent skill (`course-flake-generator`), our architecture transforms general-purpose AI assistants into domain experts capable of generating `flake.nix` configurations from natural language assignment descriptions. To eliminate package hallucinations, the pipeline integrates a Model Context Protocol (`mcp-nixos`) server that queries live NixOS package registries

in real time. Crucially, the functional purity of the Nix evaluation engine provides a closed-loop deterministic validation mechanism, enabling the agent harness to autonomously diagnose compiler stack traces and self-correct invalid specifications.

B. Synthesis of Research Questions

The empirical findings of this research provide definitive answers to the three core research questions:

- **RQ1 (Agentic Synthesis Accuracy):** AI agents can accurately infer system-level dependencies and generate valid `flake.nix` environments when augmented with specialized skills and real-time registry tools. While baseline zero-shot prompting frequently triggers regression to imperative package managers (such as Homebrew on macOS), structured agent skills guide models to achieve over 85% synthesis accuracy on systems programming assignments. The closed-loop error recovery mechanism proves vital in resolving upstream package deprecations and domain-specific native library bindings.
- **RQ2 (Clean OS Reproducibility):** A locally valid flake is not, by itself, evidence that the environment will work for a student who lacks the authoring machine's state. RQ2 therefore replays every RQ1-passing flake and lock file in a fresh, unmounted Nix Docker container. Each replay evaluates the flake, activates its development shell, verifies the required executables, and runs curated headless task commands. The resulting clean-container reproduction rate, together with classified failures in evaluation, activation,

dependency availability, or task execution, provides a stricter measure of reproducibility than a single host-side validation pass.

- **RQ3 (Student Onboarding Ergonomics):** Single-command activation via `nix develop` reduces cognitive friction during student onboarding compared to manual imperative installation. While declarative setup incurs an initial one-time overhead for installing the Nix package manager and realizing the development shell, imperative multi-step instructions expose students to command-syntax misinterpretations and skipped configuration steps (such as shell profile sourcing) that silently break the environment.

C. Discussion: Operational Constraints and Deployment Limitations

Beyond the empirical synthesis, clean-replay, and student onboarding evaluations, deploying Nix Flake environments in live classroom settings introduces theoretical operational constraints and supply-chain considerations documented in prior literature and official Nix specifications. This section examines the upstream availability, package lifecycle maintenance, binary-cache coverage, and trust-boundary limitations that instructors must account for when adopting declarative environments.

1) Adopting Nix: Nix is best utilized if the majority of the department mandated that courses use it. If only a few sections were to use nix,

The inclusion of a single flake.nix file in a project

[40]

2) Operational Availability and Maintenance Conditions: The initial macOS VM image that we used for our RQ3 test was Sonoma, which was released in 2023. Installation via brew took more than an hour and exceeded the time that student A had available. This is due to brew compiling all packages and their respective dependencies from source. According to brew's FAQ [41] pre-built binaries, also called bottles, are only provided for versions of MacOS that brew supports.

A pinned `flake.lock` separates ordinary classroom activation from dependency maintenance: students activate a committed lock file rather than invoking `nix flake update` during a lab. Consequently, classroom readiness within an instructional time budget depends on normal network operation, input-source availability, primary cache availability, and package lifecycle stability.

Nix Flakes make dependencies explicit, but they do not eliminate dependence on the services and maintainers that supply those dependencies. Nixpkgs inputs are commonly fetched from GitHub. A GitHub outage can therefore prevent an instructor from refreshing a flake input and can prevent a new machine from fetching a locked input that is not already available locally. These are different operational events: a committed lock file protects ordinary coursework from unintended version drift, whereas updating the lock file remains an instructor maintenance activity that depends on upstream availability.

The breadth of Nixpkgs also creates a maintenance risk. Package definitions may become deprecated, renamed, broken, or removed when upstream software is abandoned or when no contributor continues to maintain its Nix expression [42].

Pinning a course environment provides a stable term-level target, but it does not remove the need to review package availability before the next term or before updating a lock file. Instructors should therefore establish a refresh schedule, retain a tested lock file for the current offering, and identify a supported alternative when a course depends on a narrowly maintained package.

Finally, binary-cache misses turn an availability event into a student-facing time cost. A departmental cache or an approved external substituter can reduce that risk, but it must be governed as part of the course's trusted computing base. Classroom dependability is therefore shaped by a combination of input availability, maintained package definitions, bounded setup time, and an explicit cache-trust policy rather than by the syntactic validity of the `flake.nix` file alone.

3) Security Considerations and Binary Cache Trust Boundaries: Determinate Systems is a for profit company that builds tools for the Nix ecosystem. They also build open source tools for secure usage of Nix.

Nix normally realizes a required store path by downloading a signed substitute from a binary cache; when no acceptable substitute is available, it may instead build the path locally from source. This fallback preserves the possibility of realization but can be unsuitable for a classroom when compilation exceeds the time available for a laboratory session. Thus, cache availability is an operational reliability concern rather than merely a performance optimization.

Nix can use additional binary caches, known as substituters, to improve availability. A substitute is accepted only when its signature corresponds to a

configured trusted public key. This signature mechanism provides an integrity boundary, but adding an external cache and its key is a supply-chain decision: it authorizes that cache operator to provide binaries that students will execute. A course should therefore use only institutionally approved cache URLs and keys, document them in the course environment, and avoid asking students to trust arbitrary caches [1].

D. Limitations and Future Work

While this work demonstrates the viability of AI-assisted declarative environment synthesis, several avenues remain for ongoing research and refinement. The input data generated for testing was simple enough for Gemini models to succeed in the experiment condition where the Nix MCP server was withheld; it therefore does not establish the value of MCP retrieval for more difficult or less conventional requirements.

Although flakes guarantee reproducibility, the long-term longevity and upstream maintenance of pinned flakes remain important considerations. A longitudinal study observing real student cohorts across successive course offerings would provide valuable empirical insights into lifecycle maintenance.

Second, while the `mcp-nixos` server provides current package metadata, further research should compare experimenter-controlled MCP access across tasks with obscure, recently changed, or platform-specific dependencies. Such a study could identify when MCP retrieval improves synthesis success or reduces diagnostic iterations relative to native Nix CLI discovery alone.

Third, popular IDEs and text editors, such as JetBrain's offerings and VSCode, often have functionality that depend on programs to be present in path and do not invoke devShells themselves, having. A semester-long experiment on student experiencde with Nix could provide insights as to the barriers exist.

By bridging artificial intelligence with purely functional development environments, computer science education can bypass the fragility of manual configuration in favor of declarative development environment.

Appendix A

Appendix Title Here

A. Disclosure of LLM use

Agent harnesses, specifically Antigravity CLI was extensively used not only for the experiments conducted, but also the creation of the scripts that performed the experiment and present the results.

In the process of writing the paper, generative AI tools were used to draft paragraphs from bulleted lists of thoughts.

1) AI-Generated Artifacts: The following student onboarding guide for activating a nix environment (`nix_installation_guide.pdf`) was generated with AI assistance and provided during the benchmark and student onboarding evaluations .

Student Guide: Nix Environment Setup

Setting up reproducible course development shells with Nix and Flakes

1 Introduction

In this course, we use **Nix** with **Flakes** to provide a completely pre-configured, reproducible programming environment. Nix ensures that every student runs the identical compiler, runtime, libraries, and build tools as the instructor and grading infrastructure—eliminating the dreaded "it works on my machine" problem without requiring heavyweight virtual machines.

Platform Support

- **macOS** (Apple Silicon & Intel): Fully supported natively via Terminal.
- **Linux** (Ubuntu, Debian, Fedora, Arch, etc.): Fully supported natively.
- **Windows (via WSL2)**: Nix requires Linux. Windows students must use the **Windows Subsystem for Linux (WSL2 with Ubuntu)**. Open PowerShell as Administrator, execute `wsl -install -d Ubuntu`, restart your PC if prompted, and run all steps below inside your WSL Ubuntu terminal.

2 Step 1: Install Nix

Choose **Option A** (strongly recommended) or **Option B**.

Option A (Recommended): Determinate Systems Graphical Installer

The **Determinate Systems Installer** is the easiest, most reliable way to install Nix. It provides a user-friendly graphical installer wizard, automatically enables modern features (`flakes` and `nix-command`), configures system daemons, supports macOS security settings out of the box, and includes an uninstaller.

Recommended: Graphical Installation Steps

1. In your web browser, navigate to: https://determinate.systems/install/
2. Click the download button for your operating system to download the graphical installer package (e.g., the `.pkg` installer on macOS).
3. Double-click the downloaded file in your `Downloads` folder to launch the graphical installation wizard.
4. Follow the on-screen prompts, click **Continue** / **Install**, and enter your system administrator password when prompted.
5. Once the installer shows the success screen, click **Close**.

Option B (Alternative): Official Nix Installer + Flakes Configuration

If you choose the official upstream installer from the NixOS Foundation, you must explicitly enable experimental features (`nix-command` and `flakes`) in your user configuration file.

1. Run the official multi-user daemon installer:

```
sh <(curl -L https://nixos.org/nix/install) --daemon
```

Required: Enable Flakes in User Configuration

Unlike the Determinate installer, the official installer **does not** enable Flakes by default. You must configure `~/.config/nix/nix.conf`.

2. Create your Nix user configuration directory:

```
mkdir -p ~/.config/nix
```

3. Append the experimental features directive:

```
echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf
```

Note: You can inspect this file with `cat ~/.config/nix/nix.conf` to ensure it contains:
`experimental-features = nix-command flakes`

3 Step 2: Restart Your Terminal

After installation completes:

1. **Close your terminal completely and open a fresh terminal window.** This ensures your shell profile reloads the updated system `$PATH` pointing to Nix.
2. Verify that the Nix CLI is ready:

   ```
   nix --version
   ```

   You should see output similar to `nix (Nix) 2.x.y`.

4 Step 3: Navigate to Your Assignment Directory

Download or clone your assignment repository onto your machine. In your terminal, use `cd` to enter the assignment directory where `flake.nix` is located:

```
cd /path/to/your/assignment-folder
```

Confirm that the `flake.nix` configuration file exists in the directory:

```
ls -la flake.nix
```

What is flake.nix?

`flake.nix` defines all dependencies, compilers, libraries, test suites, and environment variables needed for the assignment.

5 Step 4: Launch the Development Environment (`nix develop`)

Once you have navigated into the assignment directory containing `flake.nix`, invoke:

```
nix develop
```

What happens when you run this command?

- **First time:** Nix evaluates `flake.nix`, downloads all pinned dependencies and tools into the isolated `/nix/store`, and configures your environment. This initial run may take a few minutes depending on your network speed.
- **Subsequent runs:** Re-entering the shell takes less than a second because all packages are already cached locally.
- **Isolated Development Shell:** You are dropped into a subshell with all project tools loaded directly into your `$PATH`. None of these tools interfere with your personal system configuration.

Working and Exiting

Inside the shell, run your project build, test, and execution commands according to your assignment instructions. When you are finished working on the assignment, exit back to your regular shell:

```
exit
```

You can also press `Ctrl + D` to exit.

6 Summary Reference Card

Action	Command
Install Nix (Recommended)	Download & run the graphical installer from https://determinate.systems/install/
Install Nix (Official)	`sh <(curl -L https://nixos.org/nix/install) -daemon`
Enable Flakes (Official)	`mkdir -p ~/.config/nix && echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf`
Verify Installation	Restart terminal, then run: `nix -version`
Enter Assignment Shell	`cd /path/to/assignment` *(where flake.nix is)* `nix develop`
Exit Assignment Shell	`exit` *(or press Ctrl+D)*

2) AI-Generated Input: The following benchmark dataset of synthetic natural language assignment instructions and expected dependency assertions was generated with AI assistance to evaluate RQ1 across the Software Development and Data Science curriculum tracks.

{"track":"computer science","instruction":"Install Java JDK 17 (includes the JRE). Set JAVA_HOME to the JDK install directory (not a separate JRE folder) and add $JAVA_HOME/bin to your PATH. Install Visual Studio Code, then install the Java extension pack (Language Support for Java by Red Hat, Debugger for Java, Java Test Runner, Maven for Java). Restart VS Code and verify with `java -version` and `javac -version`.","dependency_list":["java","javac","code"]}
{"track":"computer science","instruction":"Install the Flutter SDK and ensure `flutter` is on your PATH. Run `flutter doctor` and follow the prompts to install/fix any missing platform toolchains it reports.","dependency_list":["flutter","dart"]}

{"track":"computer science","instruction":"Install Python 3.12 and the `uv` package manager. In your project directory, run `uv add` for the following packages: flask and flask-restful, then `uv sync`. Verify with `python3 --version` and `uv --version`.","dependency_list":["python3","uv"]}
{"track":"computer science","instruction":"Install GCC (C/C++) and build tools. Verify with `gcc --version`, `g++ --version`, `cmake --version`, and `make --version`, then build the project using the provided build instructions.","dependency_list":["gcc","g++","cmake","make"]}
{"track":"computer science","instruction":"Install Clang (C/C++) and build tools. Verify with `clang --version`, `clang++ --version`, `cmake --version`, and `make --version`, then build the project using the provided build instructions.","dependency_list":["clang","clang++","cmake","make"]}
{"track":"computer science","instruction":"Install Node.js (LTS) so you have `node` and `npm`. Verify with `node --version` and `npm --version`, then run `npm install` in the project directory.","dependency_list":["node","npm"]}
{"track":"computer science","instruction":"Install Rust using rustup, then ensure `rustc` and `cargo` are available. Verify with `rustc --version` and `cargo --version`, then run `cargo build`.","dependency_list":["rustup","rustc","cargo"]}
{"track":"computer science","instruction":"Install Go and confirm the `go` command is available. Verify with `go version`, then run `go test ./...` in the project directory.","dependency_list":["go"]}
{"track":"computer science","instruction":"Install the .NET SDK (not just the runtime). Verify with `dotnet --info`, then run `dotnet restore` and `dotnet build`.","dependency_list":["dotnet"]}
{"track":"computer science","instruction":"Install Ruby and Bundler. Verify with `ruby --version` and `bundle --version`, then run `bundle install`.","dependency_list":["ruby","bundle"]}

```
{"track":"data science","instruction":"Install R (latest stable). Ensure the `R` executable is
    on your PATH, then verify with `R --version` and starting an interactive session with `R
    `.","dependency_list":["R"]}
{"track":"data science","instruction":"Install RStudio Desktop and launch it once to confirm it
     detects your R installation. If it does not, configure RStudio to point to the installed R
     version in its settings.","dependency_list":["R","rstudio"]}
{"track":"data science","instruction":"Install MySQL Server, start the database service, and
    set a root password. Verify by connecting with `mysql -u root -p` and running `SELECT
    VERSION();`.","dependency_list":["mysqld","mysql"]}
{"track":"data science","instruction":"Install MySQL Workbench (GUI client). Create a new
    connection to your local MySQL server and test the connection using the credentials you set
    .","dependency_list":["mysql-workbench","mysqld","mysql"]}
```


Listing 6. Natural Language Instruction Benchmark Inputs

Appendix B

Appendix Title Here

[Appendices follow the last page of the text. Each appendix should be labeled, either at the top or on a proceeding blank page, as Appendix A, Appendix B, etc. The sequence of the appendices follows the order in which they were first introduced in the main body of the text. Each appendix needs to be labeled and named in the main body of the text for it to be included in the appendix section.]

References

[1] D. Eelco, "Nix: A Safe and Policy-Free System for Software Deployment."

[2] E. Dolstra, *The Purely Functional Software Deployment Model*. s.n.

[3] OS as Code - My Experience of NixOS. This Cute World. [Online]. Available: https://thiscute.world/en/posts/my-experience-of-nixos/

[4] G. Stoker, T. Arnold, and P. Maxwell, "Using virtual machines to improve learning and save resources in an introductory IT course," pp. 91–96.

[5] S. Valstar, W. G. Griswold, and L. Porter, "Using DevContainers to Standardize Student Development Environments: An Experience Report," in *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*. ACM, pp. 377–383. [Online]. Available: https://dl.acm.org/doi/10.1145/3341525.3387424

[6] D. J. Malan, "Containerizing CS50: Standardizing Students' Programming Environments," in *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, ser. ITiCSE 2024. Association for Computing Machinery, pp. 534–540. [Online]. Available: https://dl.acm.org/doi/10.1145/3649217.3653567

[7] D. P. Harvie, J. R. Cody, C. Morrell, and T. T. Estes, "Using Virtual Machines to Enhance the Educational Experience in an Introductory Computing Course," in *Proceedings of the 20th Annual SIG Conference on*

Information Technology Education. ACM, pp. 28–32. [Online]. Available: https://dl.acm.org/doi/10.1145/3349266.3351401

[8] D. Holland, W. Josephson, K. Magoutis, M. Seltzer, C. Stein, and A. Lim, "Research issues in No-Futz computing," in *Proceedings Eighth Workshop on Hot Topics in Operating Systems*, pp. 106–110. [Online]. Available: https://ieeexplore.ieee.org/document/990069/

[9] W. Du, J.-M. Gratien, R. Gayno, and B. Raffin. Nix to the Rescue for a Reproducible HPC-AI Software Stack. [Online]. Available: http://arxiv.org/abs/2607.26688

[10] Nix is a better Docker image builder than Docker's image builder. [Online]. Available: https://xeiaso.net/talks/2024/nix-docker-build/

[11] E. Dolstra and A. Hemel, "Purely Functional System Configuration Management."

[12] E. Dolstra, A. Löh, and N. Pierron, "NixOS: A purely functional Linux distribution," vol. 20, no. 5–6, pp. 577–615. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0956796810000195/type/journal_article

[13] Bandt, "Assessment of Reproducibility in Nix." [Online]. Available: https://zenodo.org/records/17372811

[14] J. Malka, S. Zacchiroli, and T. Zimmermann, "Reproducibility of Build Environments through Space and Time," in *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering:*

New Ideas and Emerging Results. ACM, pp. 97–101. [Online]. Available: https://dl.acm.org/doi/10.1145/3639476.3639767

[15] (5) Building Reproducible, Interactive C++ Learning Environments with Nix Flakes and Pixi | LinkedIn. [Online]. Available: https://www.linkedin.com/pulse/building-reproducible-interactive-c-learning-nix-flakes-hasanovi%C4%87-tol9e/

[16] Modularity and Lazy Evaluation in Nixos: A Functional Approach to Operating System Configuration. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10991248?casa_token=SRx031-v1FkAAAAA:nqGmgm6Rbw4Um5tOoherapYaYndJ8JH5xvF-M0wRX4RuwscLFky9oG5f5RfFGh5UmHQfOPx633eWlQ

[17] M. Hausch, S. Hauser, and B. Uekermann, "Improving reproducibility of scientific software using Nix/NixOS: A case study on the preCICE ecosystem," vol. 83. [Online]. Available: http://eceasst.org/index.php/eceasst/article/view/2613

[18] Q. Meng, Y. Wang, L. Chen, Y. Li, W. Wu, W. Jiang, Q. Wang, C. Lu, Y. Gao, Y. Wu, and Y. Hu. Agent Harness for Large Language Model Agents: A Survey. [Online]. Available: https://www.preprints.org/manuscript/202604.0428

[19] M. A. Forment, M. J. C. Guerrero, F. J. García-Peñalvo, and J. Pereira. The Scaffolding Matters More Than the Interface: A Controlled Comparison of MCP and CLI Tool Use Across Seven Agent Scaffoldings, Five Language Models, and One Software Task. [Online]. Available: http://arxiv.org/abs/2608.08654

[20] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. M. Park, G. S. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, "IaC-Eval: A Code Generation Benchmark for Cloud Infrastructure-as-Code Programs."

[21] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun. Deployability-Centric Infrastructure-as-Code Generation: Fail, Learn, Refine, and Succeed through LLM-Empowered DevOps Simulation. [Online]. Available: https://arxiv.org/abs/2506.05623

[22] Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities. [Online]. Available: https://arxiv.org/html/2501.19012v1

[23] K. G. Srivatsa, S. Mukhopadhyay, G. Katrapati, and M. Shrivastava, "A Survey of using Large Language Models for Generating Infrastructure as Code."

[24] Agent Skills. Claude Platform Docs. [Online]. Available: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

[25] G. Ling, S. Zhong, and R. Huang. Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality. [Online]. Available: https://arxiv.org/abs/2602.08004

[26] X. Hou, Y. Zhao, S. Wang, and H. Wang. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. [Online]. Available: http://arxiv.org/abs/2503.23278

[27] D. Bork and P. Langer, "Language Server Protocol: An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools," vol. 18, pp. 9:1–16. [Online]. Available: https://emisa-journal.org/emisa/article/view/320

[28] P. Gupta, A. Gabani, C. Nelson, and Y. Shoshitaishvili. CTF Archive: Capture, Curate, Learn Forever. [Online]. Available: http://arxiv.org/abs/2512.01233

[29] J. Malka, S. Zacchiroli, and T. Zimmermann, "Does Functional Package Management Enable Reproducible Builds at Scale? Yes." in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, pp. 775–787. [Online]. Available: https://ieeexplore.ieee.org/document/11025777/

[30] A. Kate, "Generative AI for Infrastructure as Code (IaC): Automating DevOps Configuration, Scripting, and Workflow Management with LLMs."

[31] L. Beurer-Kellner, A. Kudrinskii, M. Milanta, K. B. Nielsen, H. Sarkar, and L. Tal. Technical Report: Exploring the Emerging Threats of the Agent Skill Ecosystem. [Online]. Available: http://arxiv.org/abs/2605.28588

[32] Q. Wu, S. Das, M. Amani, A. Nag, S. Lee, K. P. Gummadi, A. Ravichander, and M. B. Zafar. To Call or Not to Call: A Framework to Assess and Optimize LLM Tool Calling. [Online]. Available: http://arxiv.org/abs/2605.00737

[33] Content-Addressing File System Objects - Nix 2.35.2 Reference Manual. [Online]. Available: https://nix.dev/manual/nix/2.35/store/file-system-object/content-address#serial-nix-archive

[34] GitHub Actions Hit Three-Nines Failure; One August Outage Consumed Year's Downtime Budget. Tech Times. [Online]. Available: https://www.techtimes.com/articles/324820/20260818/github-actions-hit-three-nines-failure-one-august-outage-consumed-years-downtime-budget.htm

[35] Binary Cache - Official NixOS Wiki. [Online]. Available: https://wiki.nixos.org/wiki/Binary_Cache#Using_a_binary_cache

[36] Garnix Blog: Stop trusting Nix caches. [Online]. Available: https://garnix.io/blog/stop-trusting-nix-caches/

[37] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating Large Language Models Trained on Code. [Online]. Available: http://arxiv.org/abs/2107.03374

[38] S. Gooding. Devenv Faces Backlash Over AI-Driven Telemetry in Version 1.4, Prompting Feature Removal. [Online]. Available: https://socket.dev/blog/devenv-

faces-backlash-over-ai-driven-telemetry-in-version-1-4-1

[39] Nix is faster than Docker build. Dwarves Memo. [Online]. Available: https://memo.d.foundation/topics/devbox/research/nix-is-faster-than-docker-build

[40] E. T. Straub, "Understanding Technology Adoption: Theory and Future Directions for Informal Learning," vol. 79, no. 2, pp. 625–649. [Online]. Available: https://doi.org/10.3102/0034654308325896

[41] FAQ (Frequently Asked Questions). Homebrew Documentation. [Online]. Available: https://docs.brew.sh/FAQ

[42] Tracking: Packages to be orphaned (please consider adopting) · Issue #576945 · NixOS/nixpkgs. GitHub. [Online]. Available: https://github.com/NixOS/nixpkgs/issues/576945